

Credit for Prior Learning AI Assistant

User Manual

Version 1.0

December 2025

Developed by: Team 06, Northeastern University

Technical Lead: Ummara Ali

Sponsor: IBM Research & College of Professional Studies

<https://github.com/ummaraali2/Credit-for-Prior-Learning-AI-Assistant>

Table of Contents

Table of Contents	2
Overview	3
Product Description	3
System Architecture	3
Presentation Layer	3
Application Layer	3
Data Layer	3
Key Features	4
Student Portal	4
Faculty Portal	4
AI Capabilities	4
Supported Application Types	4
Eligible Courses	5
Getting Started	6
System Requirements	6
End Users (Students and Faculty)	6
System Administrators	6
Installation	6
Step 1: Clone Repository	6
Step 2: Install Dependencies	7
Step 3: Environment Configuration	7
Step 4: Database Initialization	7
Step 5: Service Startup	7
Step 6: Verification	8
Initial Configuration	8
For Students	8
For Faculty	8
How to Use	9
Student Portal – Submitting Applications	9
Step 1: Initiate Conversation	9
Step 2: Select Application Type	9
Step 3: Provide Detailed Information	9
Step 4: Upload Supporting Documents	9
Step 5: Review AI Preliminary Assessment	10
Step 6: Submit Application	10
Step 7: Track Application Status	10
Faculty Portal – Evaluating Applications	11
Step 1: Access Dashboard	11
Step 2: Review Application	11
Step 3: Make Decision	11
Step 4: Submit Decision	12
Advanced Features	12

Watson Assistant Configuration	13
Custom Extensions	13
Creating a Custom Extension	13
LLM Extension Configuration	13
Action Steps and LLM Looping	13
Configuring Looping Actions	13
Session Variable Management	14
watsonx.data and Apache Iceberg Configuration	15
Overview	15
Setting Up watsonx.data	15
Iceberg Table Schema	15
Querying Iceberg Tables	15
RAG Pipeline Deployment	17
Overview	17
Deploying the RAG Notebook as a Service	17
Step 1: Prepare the Notebook	17
Step 2: Create Deployment Space	17
Step 3: Deploy as Online Function	17
Step 4: Integrate with Application	17
Troubleshooting	19
Common Issues	19
Chatbot Not Responding	19
File Upload Failures	19
Document Processing Delays	19
Application Status Not Updating	19
Vector Search Returns No Results	20
Error Messages Reference	20
Frequently Asked Questions	20
Support and Maintenance	22
Contact Information	22
Technical Support	22
Academic Advising	22
Development Team	22
Documentation Resources	22
System Updates	22
Update Schedule	22
Update Procedure for Administrators	22
Backup and Recovery	23
Security and Compliance	23
Data Protection	23
FERPA Compliance	23
Incident Response	23
Document Information	24
Change History	24

Overview

Product Description

The Credit for Prior Learning (CPL) AI Assistant is an intelligent automation system designed to evaluate student learning experiences for academic credit at Northeastern University's College of Professional Studies. The system leverages IBM watsonx.ai, Watson Assistant, Milvus vector database, and Apache Iceberg to provide comprehensive workflow automation from application submission to credit award.

This solution streamlines the traditionally manual and time-intensive CPL evaluation process by incorporating artificial intelligence capabilities that assist both students in preparing their applications and faculty in conducting thorough evaluations.

System Architecture

The CPL Bot implements a robust three-tier architecture designed for scalability, maintainability, and security.

Presentation Layer

- Watson Assistant embedded widget for conversational interface
- HTML5, CSS3, and JavaScript frontend for student and faculty portals
- Responsive web design accessible via modern browsers

Application Layer

The application layer consists of two primary components working in tandem:

Node.js Gateway (Express.js):

- Handles HTTP requests and Cross-Origin Resource Sharing (CORS)
- Manages file uploads via Multer middleware
- Routes requests to the Python backend
- Serves static frontend assets

Python AI Backend (Flask):

- Document processing with PyPDF2 and python-docx
- Text extraction and intelligent chunking with context preservation
- Embedding generation via watsonx.ai API
- Vector storage and retrieval using PyMilvus
- RAG pipeline orchestration with LangChain

Data Layer

Milvus Vector Database:

- High-dimensional embedding storage
- HNSW index with L2 distance metric for efficient similarity search

The screenshot displays the IBM watsonx web interface. The top navigation bar shows the user is logged in as Allison Ruda. The main content area is titled 'Projects / CPL Chatbot / CPL Documents v5'. On the left, there are tabs for 'Details', 'Sample questions', and 'Test'. The 'Details' tab is active, showing a table of configuration parameters for the vector index. On the right, a sidebar titled 'About this asset' provides additional information about the vector index, including its name, description, and asset details.

Details	
Vector store	watsonx.data
Vector store connection	milvus
Embeddings model	ibm/slate-125m-english-rtrvr-v2
Vector index ID	847e4983-4868-4823-a1db-d02b42f1d706
Search settings	Top K: 15 Edit search settings
Milvus database	default
Milvus collection	cpl_documents_v5 258 document chunks
Milvus collection schema	Document name field: document_name Text field: text Page number field: page

About this asset

Name
CPL Documents v5
Vector index

Description
What's the purpose of this asset?

Asset details
Vector store
watsonx.data
Connection
[milvus](#)
Milvus database
default
Milvus collection
cpl_documents_v5 (258 document chunks)

Last modified
3 d ago by Ummara Ali Syeda
Created on
3 d ago by Ummara Ali Syeda
[Open in Prompt Lab](#)

IBM Cloud Object Storage:

- Original document preservation
- S3-compatible API for seamless integration

IBM watsonx.data (Apache Iceberg):

- Presto SQL interface for analytics
- Metadata storage and querying with ACID compliance

Student Portal

- ## Faculty Portal

- Page 6 of 31

- Batch operations for high-volume evaluation periods

AI Capabilities

- RAG architecture retrieves relevant course syllabi from vector store
- Semantic similarity search across student submissions
- Automated competency mapping against learning outcomes
- Intelligent routing to appropriate department faculty
- Context-aware chunking that preserves document meaning

Supported Application Types

Experience-Based CPL: Evaluation of professional work experience against course learning outcomes.

Prior Coursework CPL: Assessment of courses completed at other accredited institutions.

Credit Transfer: Processing of official transcripts for course equivalency determination.

Eligible Courses

The CPL system currently supports Project Management (PJM) courses from Northeastern University's College of Professional Studies. Please contact your academic advisor to verify eligibility for specific courses and programs.

Getting Started

System Requirements

End Users (Students and Faculty)

Minimum Requirements:

- Modern web browser: Google Chrome, Mozilla Firefox, Safari, or Microsoft Edge (latest versions recommended)
- Stable internet connection
- JavaScript enabled
- Cookies and local storage enabled

Supported File Formats:

- PDF documents (.pdf)
- Microsoft Word documents (.docx)
- Plain text files (.txt)

System Administrators

Server Requirements:

- Node.js (version 18 or higher)
- Python (version 3.9 or higher)
- Operating System: Linux (Ubuntu recommended), macOS, or Windows Server
- Adequate RAM and storage allocation based on expected usage
- Network access with required ports available

External Service Requirements:

- IBM Cloud account with active subscriptions
- watsonx.ai service instance
- Watson Assistant service instance
- Cloud Object Storage bucket with S3 API enabled
- watsonx.data workspace with Presto access
- Milvus cluster (self-hosted or cloud-managed)

Installation

Step 1: Clone Repository

Access the project repository and clone it to your local environment using the following command:

```
git clone
https://github.com/ummaraali2/Credit-for-Prior-Learning-AI-Assistant.git
```

Step 2: Install Dependencies

Node.js packages:

```
npm install
```

Required packages include express, cors, multer, form-data, and dotenv.

Python packages:

```
pip install -r requirements.txt
```

Required packages include Flask, ibm-watsonx-ai, pymilvus, ibm-cos-sdk, prestodb, PyPDF2, python-docx, langchain, and python-dotenv.

Step 3: Environment Configuration

Create a .env file in the project root directory and configure the following environment variables with your specific credentials and endpoints:

- IBM watsonx.ai Configuration (API key, service URL, project ID)
- Milvus Vector Database (connection ID, host, port, credentials)
- IBM Cloud Object Storage (API key, instance ID, endpoint, bucket name)
- IBM watsonx.data / Apache Iceberg (host, port, user, password, catalog, schema, table)

- Watson Assistant Configuration (integration ID, region, service instance ID)

Sample .env configuration file

```
# IBM watsonx.ai Configuration
WATSONX_AI_APIKEY=your_watsonx_api_key
WATSONX_AI_SERVICE_URL=https://us-south.ml.cloud.ibm.com
WATSONX_AI_PROJECT_ID=your_project_id

# Milvus Vector Database
MILVUS_CONNECTION_ID=your_milvus_connection_id
MILVUS_HOST=your_milvus_host
MILVUS_PORT=32668
MILVUS_USERNAME=your_username
MILVUS_PASSWORD=your_password

# IBM Cloud Object Storage
COS_API_KEY=your_cos_api_key
COS_INSTANCE_ID=your_cos_instance_id
COS_ENDPOINT=https://s3.us-south.cloud-object-storage.ibmcloud.com
COS_BUCKET_NAME=cpl-documents

# IBM watsonx.data (Iceberg)
WATSONX_DATA_HOST=your_presto_host
WATSONX_DATA_PORT=30670
WATSONX_DATA_USER=ibmlhapikey_your_email
WATSONX_DATA_PASSWORD=your_api_key
ICEBERG_CATALOG=iceberg_data
ICEBERG_SCHEMA=cpl_schema
ICEBERG_TABLE=cpl_requests
```

Step 4: Database Initialization

Create Milvus Collection:

Navigate to the backend scripts directory and execute the collection creation script. This creates the required collection with appropriate dimensions matching the IBM embedding model output, HNSW index type, and L2 metric.

Create Iceberg Table:

Execute the CREATE-TABLE.sql script in the watsonx.data Presto interface to create the metadata storage table with all required fields including request tracking, student information, and audit fields.

Milvus collection creation output

```
CREATE TABLE IF NOT EXISTS iceberg_data.cpl_schema.cpl_requests (

    request_id VARCHAR,

    document_id VARCHAR,

    student_name VARCHAR,
```

```

nuid VARCHAR,

request_type VARCHAR,

target_course VARCHAR,

status VARCHAR,

credits_awarded DECIMAL(4,2),

advisor_notes VARCHAR,

submitted_date TIMESTAMP,

updated_date TIMESTAMP,

updated_by VARCHAR,

document_count INT

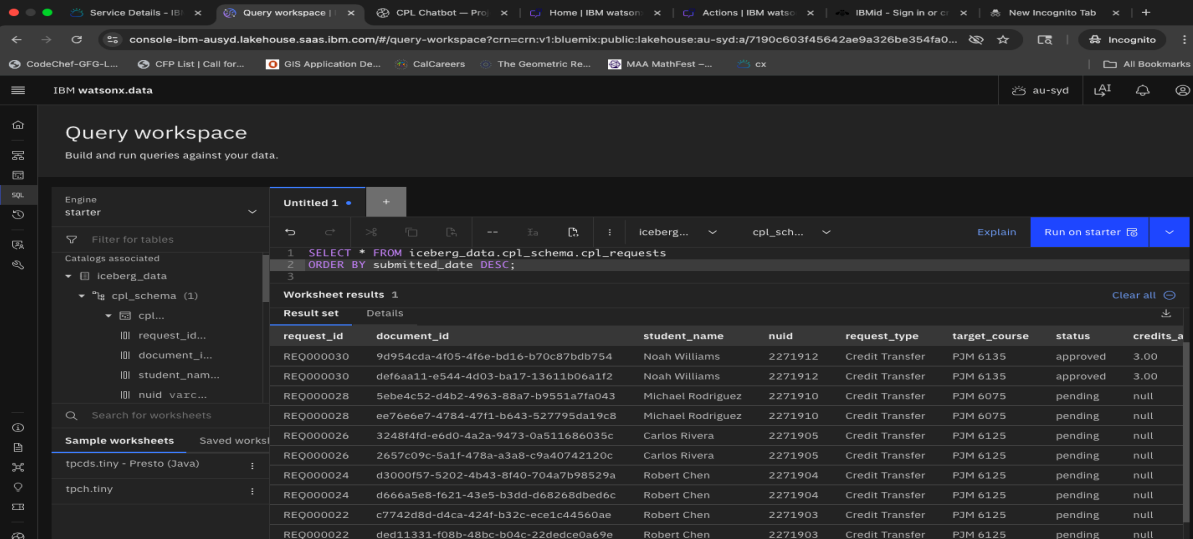
) WITH (

format = 'PARQUET',

partitioned_by = ARRAY['status']

);

```



The screenshot displays the IBM Watsonx Data Query workspace. The left sidebar shows the 'Engine starter' and 'Catalogs associated' section, with 'iceberg_data' and 'cpl...' visible. The main area shows a query editor with the following SQL query:

```

1 SELECT * FROM iceberg_data.cpl_schema.cpl_requests
2 ORDER BY submitted_date DESC;
3

```

The 'Run on starter' button is visible. Below the query editor, the 'Worksheet results' section shows a table of results:

request_id	document_id	student_name	nuid	request_type	target_course	status	credits_a
REQ000030	9d954cda-4f05-4f6e-bd16-b70c87bdb754	Noah Williams	2271912	Credit Transfer	PJM 6135	approved	3.00
REQ000030	def6aa11-e544-4d03-ba17-13611b06a1f2	Noah Williams	2271912	Credit Transfer	PJM 6135	approved	3.00
REQ000028	5ebe4c52-d4b2-4963-88a7-b9551a7fa043	Michael Rodriguez	2271910	Credit Transfer	PJM 6075	pending	null
REQ000028	ee76e6e7-4784-47f1-b643-527795da19c8	Michael Rodriguez	2271910	Credit Transfer	PJM 6075	pending	null
REQ000026	3248f4fd-eed0-4a2a-9473-0a511686035c	Carlos Rivera	2271905	Credit Transfer	PJM 6125	pending	null
REQ000026	2657c09c-5a1f-478a-a3a8-c9a40742120c	Carlos Rivera	2271905	Credit Transfer	PJM 6125	pending	null
REQ000024	d3000f57-5202-4b43-8f40-704a7b98529a	Robert Chen	2271904	Credit Transfer	PJM 6125	pending	null
REQ000024	d666a5e8-f621-43e5-b3dd-d68268dbed0c	Robert Chen	2271904	Credit Transfer	PJM 6125	pending	null
REQ000022	c7742d8d-d4ca-424f-b32c-ecelc44560ae	Robert Chen	2271903	Credit Transfer	PJM 6125	pending	null
REQ000022	dcd11331-f08b-48bc-b04c-22dedce0a69e	Robert Chen	2271903	Credit Transfer	PJM 6125	pending	null

Step 5: Service Startup

Start both services in separate terminal windows:

Terminal 1 - Python AI Backend:

```
cd backend/servicespython watson_upload.py
```

Terminal 2 - Node.js Gateway:

```
npm start
```

Step 6: Verification

Verify the installation by accessing the health check endpoints and navigating to the application URL in your browser. Both services should report healthy status with all dependent services connected.

Initial Configuration

For Students

1. Navigate to the application URL
2. Authenticate with Northeastern University credentials via SSO integration
3. The Watson Assistant widget will appear in the bottom-right corner of the interface
4. Begin a conversation to register your profile information

Profile data collected includes: full name, Northeastern University ID (NUID), email address, phone number (optional), program of study, and expected graduation date. All data is stored securely with encryption at rest.

For Faculty

1. Request access from the system administrator
2. Receive credentials via secure university email
3. Navigate to the faculty portal
4. Complete mandatory password change on first login
5. Configure preferences including email notifications, default sort order, and department affiliation.

How to Use

Student Portal – Submitting Applications

Step 1: Initiate Conversation

Click the Watson Assistant widget icon located in the bottom-right corner of the interface. Begin with a greeting or a direct request such as:

- *"Hello"*
- *"I want to apply for CPL"*
- *"Apply for credit based on work experience"*

The bot will respond with a welcome message and request your identification (name and NUID).

Step 2: Select Application Type

Three application types are available:

Experience-Based CPL: Evaluates professional work experience against course learning outcomes. Suitable for candidates with relevant professional experience.

Prior Coursework CPL: Assesses courses completed at other accredited institutions. Requires course syllabus and transcript documentation.

Credit Transfer: Processes official transcripts for course equivalency. Official transcripts must be sent by the previous institution.

Step 3: Provide Detailed Information

For Experience-Based applications (the most common type), the bot will request the following information:

- Target course (e.g., PJM 5100 Project Management Fundamentals)
- Current or recent job title
- Employment duration
- Detailed responsibilities
- Methodologies and tools used
- Quantifiable achievements

Provide comprehensive and specific information to ensure the most accurate assessment of your qualifications.

Step 4: Upload Supporting Documents

Required Documents:

- Current resume or curriculum vitae
- Job description from employer
- Performance reviews (if available)
- Professional certifications (PMP, CAPM, CSM, etc.)

Optional Documents:

- Project plans or charters (sanitized to remove confidential information)
- Project completion reports
- Letters from supervisors

To upload documents, click the paperclip icon in the chat interface, select files from your device, and allow the system to process them. The system performs text extraction, intelligent chunking, embedding generation, and secure storage automatically.

Step 5: Review AI Preliminary Assessment

The system performs RAG-based analysis by retrieving relevant course syllabus content from the vector database, comparing your submission against learning

outcomes, and generating a competency mapping with confidence indicators. The assessment includes:

- Overall match assessment
- Competency-by-competency analysis
- Evidence citations linking to specific document sections
- Recommended credit award
- Confidence level indicator

Please note that the AI assessment is advisory only. The final decision is made by faculty reviewers.

Step 6: Submit Application

Review the application summary showing your information, target course, uploaded documents, and AI preliminary assessment. Type "SUBMIT" to finalize your application. The system generates a unique application ID for tracking purposes.

Upon submission, the following automated workflow occurs: the application is stored in the database, the routing algorithm determines the appropriate department, a faculty member is assigned based on expertise and workload, and email notifications are sent to both you and the assigned faculty reviewer.

Step 7: Track Application Status

Query your application status via the chatbot using phrases such as "Check status of [Application ID]" or "Show my applications." Application status values include:

- **SUBMITTED:** Application received, pending initial review
- **ASSIGNED:** Faculty reviewer assigned
- **UNDER_REVIEW:** Faculty actively evaluating
- **INFO_REQUESTED:** Faculty needs additional evidence
- **APPROVED:** Credit awarded, pending transcript posting
- **DENIED:** Application not approved
- **POSTED:** Credits visible on official transcript

Faculty Portal – Evaluating Applications

Step 1: Access Dashboard

Login to the faculty portal with your university credentials. The dashboard displays pending applications, in-progress evaluations, completed evaluations, and department statistics. The queue is sortable by date, priority, and course.

Step 2: Review Application

Click an application row to open the evaluation interface. The view includes:

AI Evaluation Summary: Overall recommendation with confidence level, competency analysis per learning outcome, evidence mapping with document citations, and suggested credit award.

Document Viewer: Inline PDF preview with AI-highlighted relevant sections, cross-reference links between competencies and evidence, and side-by-side syllabus comparison for coursework evaluations.

Student Information Panel: Academic history, previous CPL applications, current enrollment status, and program of study.

Step 3: Make Decision

Complete the evaluation form by selecting a decision option:

For Approval:

- Specify credits awarded
- Select credit type (Full course waiver, Partial credit, or Conditional)
- Add conditions if applicable
- Provide student feedback
- Add internal notes (not visible to student)

For Denial:

- Provide specific reasons for denial
- Offer suggestions for addressing gaps
- Recommend alternative pathways if applicable

For Information Request:

- Specify documents or clarifications needed
- Set a deadline for response

Step 4: Submit Decision

Certify your evaluation by confirming all documents have been reviewed, verifying student identity via NUID, and acknowledging compliance with institutional CPL standards. Click "Submit Decision" to finalize.

The automated post-submission workflow records the decision, updates the application status, notifies the student via email and chatbot, notifies the academic advisor, and creates an audit log entry. For approved applications, credits are posted to the transcript by the registrar.

Advanced Features

Batch Operations: Select multiple similar applications, apply consistent decision criteria, particularly useful for transfer credit evaluations.

Search and Filter: Filter by status, date range, course, or student; full-text search across application content; export results to CSV for reporting.

Analytics: Personal evaluation statistics, department-wide metrics, and processing time trends.

Demonstration

[Watch here](#)

Watsonx Assistant Configuration

Custom Extensions

Watson Assistant uses custom extensions to connect with external services and APIs. These extensions enable the conversational interface to perform complex operations such as document processing and database queries.

Creating a Custom Extension

1. Navigate to the Watson Assistant console and select your assistant
2. Go to the Integrations section and select "Build custom extension"
3. Provide a name and description for the extension
4. Upload an OpenAPI specification file defining the API endpoints
5. Configure authentication settings (API key, OAuth, or no authentication)
6. Add the extension to your assistant and configure the server URL

LLM Extension Configuration

The CPL system utilizes a custom LLM extension to connect Watson Assistant with watsonx.ai for advanced natural language processing capabilities. This extension enables semantic understanding of student queries and intelligent response generation.

1. Create an OpenAPI specification defining the watsonx.ai endpoint
2. Include request parameters for prompt, model selection, and generation parameters
3. Configure the extension with your watsonx.ai API credentials
4. Test the extension using the preview panel

Action Steps and LLM Looping

Watson Assistant Actions enable complex conversational flows with the ability to loop back to LLM extensions for iterative processing. This capability is essential for handling multi-turn conversations where context must be maintained.

Configuring Looping Actions

1. Open the Actions editor in Watson Assistant
2. Create or edit an action that requires LLM interaction
3. Add a step that calls the LLM extension with the appropriate prompt
4. Store the LLM response in a session variable

5. Add conditional logic to determine if additional LLM calls are needed
6. Configure the step to loop back to the LLM extension step when conditions are met
7. Set a maximum loop count to prevent infinite loops

Session Variable Management

Proper session variable management is critical for maintaining context across LLM interactions:

- Define session variables for storing conversation history
- Append new user inputs and LLM responses to the history variable
- Pass the accumulated context to the LLM extension on each call
- Clear session variables at appropriate conversation boundaries

watsonx.data and Apache Iceberg Configuration

Overview

IBM watsonx.data provides a data lakehouse architecture built on Apache Iceberg, offering ACID transactions, time travel capabilities, and schema evolution without downtime. The CPL system leverages this platform for metadata storage and analytical queries.

Setting Up watsonx.data

1. Access the IBM Cloud console and provision a watsonx.data instance
2. Configure the Presto engine for SQL query processing
3. Create a catalog for Iceberg tables
4. Define the schema for CPL request data
5. Configure access credentials and connection parameters

The screenshot displays the IBM watsonx.data Query workspace. The interface includes a sidebar with navigation options like 'Engine starter', 'Filter for tables', and 'Catalogs associated'. The main area shows a SQL query titled 'Untitled 1' with the following code:

```
1 SELECT * FROM iceberg_data.cpl_schema.cpl_requests
2 ORDER BY submitted_date DESC;
```

Below the query, the 'Worksheet results' section shows a table with 9 columns: request_id, document_id, student_name, nuid, request_type, target_course, status, and credits_a. The table contains 10 rows of data, including requests from Noah Williams, Michael Rodriguez, Carlos Rivera, and Robert Chen.

request_id	document_id	student_name	nuid	request_type	target_course	status	credits_a
REQ000030	9d954cda-4f05-4f6e-bd16-b70c87bdb754	Noah Williams	2271912	Credit Transfer	PJM 6135	approved	3.00
REQ000030	def6aa11-e544-4d03-ba17-13611b06a1f2	Noah Williams	2271912	Credit Transfer	PJM 6135	approved	3.00
REQ000028	5ebe4c52-d4b2-4963-88a7-b9551a7fa043	Michael Rodriguez	2271910	Credit Transfer	PJM 6075	pending	null
REQ000028	ee76e6e7-4784-47f1-b643-527795da19c8	Michael Rodriguez	2271910	Credit Transfer	PJM 6075	pending	null
REQ000026	3248f4fd-e6d0-4a2a-9473-0a511686035c	Carlos Rivera	2271905	Credit Transfer	PJM 6125	pending	null
REQ000026	2657c09c-5a1f-478a-a3a8-c9a40742120c	Carlos Rivera	2271905	Credit Transfer	PJM 6125	pending	null
REQ000024	d3000f57-5202-4b43-8f40-704a7b98529a	Robert Chen	2271904	Credit Transfer	PJM 6125	pending	null
REQ000024	d666a5e8-f621-43e5-b3dd-d68268dbed6c	Robert Chen	2271904	Credit Transfer	PJM 6125	pending	null
REQ000022	c7742d8d-d4ca-424f-b32c-ec1c44560ae	Robert Chen	2271903	Credit Transfer	PJM 6125	pending	null
REQ000022	ded11331-f08b-48bc-b04c-22dedce0a69e	Robert Chen	2271903	Credit Transfer	PJM 6125	pending	null

Iceberg Table Schema

The CPL requests table stores comprehensive metadata about each application, including student information, course details, status tracking, and evaluation results. The table is partitioned by creation month for efficient time-range queries.

Key table fields include:

- Request identifier (primary key)
- Student identification and contact information
- Course code and title
- Request type and status

- Timestamp fields for creation and updates
- Document identifiers array
- Faculty reviewer assignment
- Credit request and award values
- AI confidence indicator and metadata map

Querying Iceberg Tables

Use the Presto SQL interface to query CPL request data. Common operations include retrieving applications by student, filtering by status, and generating analytical reports. The system supports time travel queries for auditing and compliance purposes.

RAG Pipeline Deployment

Overview

The Retrieval-Augmented Generation (RAG) pipeline is the core AI component of the CPL system. It combines document retrieval from the Milvus vector database with intelligent response generation to provide accurate competency assessments.

Deploying the RAG Notebook as a Service

The RAG pipeline can be deployed as a service application using IBM watsonx.ai. This enables scalable, on-demand access to the AI capabilities.

Step 1: Prepare the Notebook

1. Open the RAG pipeline notebook in watsonx.ai Studio
2. Verify all dependencies are properly imported
3. Configure environment variables for external service connections
4. Test the notebook execution end-to-end

Step 2: Create Deployment Space

1. Navigate to Deployment Spaces in watsonx.ai
2. Create a new deployment space for the CPL application
3. Associate the required compute resources
4. Configure access permissions for the deployment

Step 3: Deploy as Online Function

1. Promote the notebook to the deployment space
2. Select "Online" deployment type for real-time inference
3. Configure scaling parameters based on expected load
4. Deploy the function and obtain the endpoint URL

5. Generate API credentials for secure access

Step 4: Integrate with Application

Update the Python backend to call the deployed RAG service endpoint instead of running the pipeline locally. This enables centralized management and scaling of the AI capabilities.

- Update environment configuration with deployment endpoint
- Implement API client for calling the deployed function
- Configure retry logic and error handling
- Test end-to-end integration with sample applications

Troubleshooting

Common Issues

Chatbot Not Responding

Symptoms: Watson Assistant widget does not respond to user input or displays error messages.

Diagnostic Steps:

- Check browser JavaScript console for CORS errors or failed API calls
- Verify Watson Assistant script loaded successfully
- Check network tab for failed API calls

Solutions:

- Clear browser cache and reload the page
- Disable browser extensions by trying incognito mode
- Verify JavaScript is enabled in browser settings
- Check network connectivity to the gateway server
- Try a different browser to isolate browser-specific issues

File Upload Failures

Error: "File too large"

- Solution: Compress PDF using Adobe Acrobat or online compression tools, or split large documents into multiple smaller files.

Error: "Unsupported file type"

- Solution: Convert document to PDF, DOCX, or TXT format. Ensure file extension matches actual file type.

Error: "Upload timeout"

- Solution: Reduce file size, try during off-peak hours, or check Python backend logs for processing errors.

Error: "Text extraction failed"

- Solution: For scanned PDFs, run OCR processing before upload. Verify the file opens correctly in its native application.

Document Processing Delays

Processing delays may occur due to large file sizes, multiple concurrent uploads, or API rate limiting. Monitor the processing status and check Python backend logs for detailed error information.

Application Status Not Updating**Possible Causes:**

- Browser cache showing stale data
- WebSocket connection dropped
- Database update latency

Solutions:

- Perform a hard refresh of the page
- Clear browser cache and reload
- Query application status directly via the API

Vector Search Returns No Results**Possible Causes:**

- Documents not yet indexed in Milvus
- Embeddings generation failed
- Collection not properly initialized

Verify the collection exists and contains data using the PyMilvus client. Check the collection indexes and test queries with sample vectors.

Error Messages Reference

Code	Message	Solution
400	Invalid request format	Check request body matches API specification
401	Authentication failed	Verify API keys in environment configuration
403	Permission denied	Check user role and permissions
404	Resource not found	Verify application ID and endpoint URL
413	Payload too large	Compress or split file
429	Rate limit exceeded	Implement exponential backoff and retry
500	Internal server error	Check Python backend logs for details
503	Service unavailable	Retry after delay, check service status

Frequently Asked Questions

Q: What embedding model does the system use?

A: The system uses IBM's slate model for embedding generation, which is optimized for retrieval tasks and produces high-dimensional embeddings suitable for semantic similarity search.

Q: How does the RAG pipeline work?

A: The RAG implementation processes student submissions by chunking documents into segments, generating embeddings, and storing them in Milvus. During evaluation, the system retrieves relevant course syllabus sections, compares them against learning outcomes, and generates competency mappings with confidence indicators.

Q: What is the optimal document format for upload?

A: PDF documents with searchable text (not scanned images) provide the best results. Microsoft Word documents are also well-supported. For scanned documents, OCR processing is recommended before upload.

Q: How long does application processing typically take?

A: Document processing typically completes within a few minutes after upload. Faculty review timelines vary based on application volume and complexity. Status updates are available through the chatbot interface.

Q: Can the system integrate with external learning management systems?

A: Yes, the system supports integration with external LMS and SIS platforms through API connections. Integration requires OAuth authentication configuration and can include webhook notifications for status changes.

Support and Maintenance

Contact Information

Technical Support

Email: ummaraali2020@gmail.com

Technical support is available for system issues, bug reports, and technical questions related to the CPL AI Assistant.

Technical Lead: Ummara Ali

GitHub Issues:

<https://github.com/ummaraali2/Credit-for-Prior-Learning-AI-Assistant/issues>

Documentation Resources

GitHub Repository:

<https://github.com/ummaraali2/Credit-for-Prior-Learning-AI-Assistant>

The repository contains complete source code, API documentation, database schemas, test suites, and additional technical documentation.

System Updates

Update Schedule

Regular Maintenance: Scheduled maintenance windows are communicated in advance via email notification.

Emergency Maintenance: Security patches and critical fixes are deployed as needed with appropriate notification when possible.

Update Procedure for Administrators

1. Backup current state (Milvus collection, Iceberg data, COS bucket)
2. Pull latest code from the repository
3. Update dependencies for Node.js and Python
4. Run database migrations if applicable
5. Restart services and verify health endpoints
6. Test critical paths including application submission and faculty workflow

Backup and Recovery

Automated backups are performed regularly to ensure data protection and business continuity. Daily backups include Milvus vectors, Iceberg table snapshots, and configuration files. Weekly full system backups provide comprehensive recovery points.

For data recovery requests or restoration from backup, contact the technical support team with the specific recovery requirements.

Security and Compliance

Data Protection

- Encryption in transit: TLS for all HTTP communications
- Encryption at rest: AES encryption for stored documents and data
- Access control: Role-based permissions for students, faculty, and administrators

FERPA Compliance

The system handles student educational records in accordance with FERPA requirements. All data access is logged for audit purposes, and the minimal necessary disclosure principle is applied throughout the system.

Incident Response

Report security concerns immediately to: ummaraali2020@gmail.com

The incident response procedure includes identification and containment, impact assessment, notification to security team, documentation, remediation, and user notification as required.

References

Python Libraries & Documentation

Core IBM SDKs

1. ibm-watsonx-ai: <https://ibm.github.io/watsonx-ai-python-sdk/>
2. ibm-watson SDK: <https://github.com/watson-developer-cloud/python-sdk>
3. ibm-cos-sdk: <https://ibm.github.io/ibm-cos-sdk-python/>
4. ibm-cloud-sdk-core: <https://github.com/IBM/python-sdk-core>

Vector Database

5. pymilvus Documentation: <https://milvus.io/docs/install-pymilvus.md>
6. pymilvus API Reference: <https://milvus.io/api-reference/pymilvus/v2.3.x/About.md>
7. Milvus Collection Schema: <https://milvus.io/docs/schema.md>
8. Milvus DataType Enum: <https://milvus.io/api-reference/pymilvus/v2.3.x/DataType.md>

Data Processing

9. pandas Documentation: <https://pandas.pydata.org/docs/>
10. numpy Documentation: <https://numpy.org/doc/stable/>
11. PyPDF2: <https://pypdf2.readthedocs.io/en/latest/>
12. pdf2image: <https://pdf2image.readthedocs.io/en/latest/>
13. pdfplumber: <https://github.com/jsvine/pdfplumber>
14. python-magic: <https://github.com/ahupp/python-magic>

LangChain Ecosystem

15. LangChain Documentation: https://python.langchain.com/docs/get_started/introduction
16. LangChain Agents: <https://python.langchain.com/docs/modules/agents/>
17. LangChain Tools: <https://python.langchain.com/docs/modules/tools/>
18. LangChain Memory: <https://python.langchain.com/docs/modules/memory/>
19. LangChain Embeddings: https://python.langchain.com/docs/modules/data_connection/text_embedding/
20. LangChain VectorStores: https://python.langchain.com/docs/modules/data_connection/vectorstores/
21. LangChain Document Loaders: https://python.langchain.com/docs/modules/data_connection/document_loaders/
22. LangChain Text Splitters: https://python.langchain.com/docs/modules/data_connection/document_transformers/
23. langchain-openai: <https://python.langchain.com/docs/integrations/platforms/openai>
24. langchain-community: <https://python.langchain.com/docs/integrations/providers/>

Web Framework

25. Flask Documentation: <https://flask.palletsprojects.com/>

- 26. Flask CORS: <https://flask-cors.readthedocs.io/>
- 27. Flask Request/Response: <https://flask.palletsprojects.com/en/2.3.x/api/#flask.Request>

HTTP & Async

- 28. requests Library: <https://requests.readthedocs.io/>
- 29. aiohttp: <https://docs.aiohttp.org/>
- 30. asyncio: <https://docs.python.org/3/library/asyncio.html>

Data Validation

- 31. Pydantic: <https://docs.pydantic.dev/latest/>
- 32. Pydantic BaseModel: <https://docs.pydantic.dev/latest/concepts/models/>
- 33. Pydantic Field Types: <https://docs.pydantic.dev/latest/concepts/fields/>

Node.js/JavaScript Libraries & Documentation

Core Backend Framework

- 34. Express.js: <https://expressjs.com/>
- 35. Express Router: <https://expressjs.com/en/guide/routing.html>
- 36. Express Middleware: <https://expressjs.com/en/guide/using-middleware.html>
- 37. Express Error Handling: <https://expressjs.com/en/guide/error-handling.html>

File Processing

- 38. pdf-parse (v1.1.1): <https://www.npmjs.com/package/pdf-parse>
- 39. pdf-parse GitHub: <https://github.com/modesty/pdf-parse>
- 40. multer: <https://www.npmjs.com/package/multer>
- 41. busboy: <https://www.npmjs.com/package/busboy>
- 42. formidable: <https://www.npmjs.com/package/formidable>

HTTP & CORS

- 43. cors: <https://www.npmjs.com/package/cors>
- 44. axios: <https://axios-http.com/docs/intro>
- 45. node-fetch: <https://www.npmjs.com/package/node-fetch>

Watson SDKs

- 46. watson-developer-cloud (Node SDK): <https://github.com/watson-developer-cloud/node-sdk>
- 47. ibm-watson/assistant-v2: <https://cloud.ibm.com/apidocs/assistant/assistant-v2?code=node>
- 48. watson-assistant npm: <https://www.npmjs.com/package/ibm-watson>

Environment & Config

- 49. dotenv: <https://www.npmjs.com/package/dotenv>

50. config: <https://www.npmjs.com/package/config>

Utilities

51. uuid: <https://www.npmjs.com/package/uuid>
52. lodash: <https://lodash.com/docs/>
53. moment.js: <https://momentjs.com/docs/>

Frontend Libraries & Documentation

Watson Assistant Web Chat

54. Watson Web Chat Widget:
<https://web-chat.global.assistant.watson.cloud.ibm.com/docs.html>
55. Web Chat API Methods:
<https://web-chat.global.assistant.watson.cloud.ibm.com/docs.html?to=api-instance-methods>
56. Web Chat Configuration:
<https://web-chat.global.assistant.watson.cloud.ibm.com/docs.html?to=api-configuration>
57. Web Chat Events:
<https://web-chat.global.assistant.watson.cloud.ibm.com/docs.html?to=api-events>
58. writeableElements API:
<https://web-chat.global.assistant.watson.cloud.ibm.com/docs.html?to=api-instance-methods#writeableelements>
59. updateUserDefinedResponse:
<https://web-chat.global.assistant.watson.cloud.ibm.com/docs.html?to=api-instance-methods#updateuserdefinedresponse>

Vanilla JavaScript

60. Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
61. FormData API: <https://developer.mozilla.org/en-US/docs/Web/API/FormData>
62. File API: <https://developer.mozilla.org/en-US/docs/Web/API/File>
63. FileReader API: <https://developer.mozilla.org/en-US/docs/Web/API/FileReader>
64. Blob API: <https://developer.mozilla.org/en-US/docs/Web/API/Blob>

HTML5 & CSS

65. HTML File Input:
<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/input/file>
66. CSS Flexbox:
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Flexible_Box_Layout
67. CSS Grid: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Grid_Layout

IBM Watson Services Documentation

Watson Assistant Deep Dive

68. Actions Step Reference:

<https://cloud.ibm.com/docs/watson-assistant?topic=watson-assistant-step-reference>

69. Variable Storage:

<https://cloud.ibm.com/docs/watson-assistant?topic=watson-assistant-manage-info#manage-info-store-vars>

70. Session History:

<https://cloud.ibm.com/docs/watson-assistant?topic=watson-assistant-manage-info#manage-info-session-history>

71. Built-in Variables:

<https://cloud.ibm.com/docs/watson-assistant?topic=watson-assistant-expression-methods-actions>

72. Array Methods:

<https://cloud.ibm.com/docs/watson-assistant?topic=watson-assistant-expression-methods-actions#expression-methods-actions-arrays>

73. String Methods:

<https://cloud.ibm.com/docs/watson-assistant?topic=watson-assistant-expression-methods-actions#expression-methods-actions-strings>

Watson Extensions & Integrations

74. Custom Extensions Guide:

<https://cloud.ibm.com/docs/watson-assistant?topic=watson-assistant-build-custom-extension>

75. OpenAPI Specification for Extensions:

<https://cloud.ibm.com/docs/watson-assistant?topic=watson-assistant-build-custom-extension#build-custom-extension-openapi-file>

76. Extension Authentication:

<https://cloud.ibm.com/docs/watson-assistant?topic=watson-assistant-add-custom-extension#add-custom-extension-auth>

watsonx.ai API Details

77. Text Generation Parameters:

<https://cloud.ibm.com/apidocs/watsonx-ai#text-generation>

78. Embedding Generation: <https://cloud.ibm.com/apidocs/watsonx-ai#embeddings>79. Model Deployment: <https://cloud.ibm.com/apidocs/watsonx-ai#deployments-create>

80. Credentials Management:

<https://dataplatform.cloud.ibm.com/docs/content/wsj/analyze-data/ml-authentication.html>

watsonx.data Specifics

81. Presto Connection Strings:

<https://cloud.ibm.com/docs/watsonxdata?topic=watsonxdata-con-presto-gs>

82. SQL Query Syntax:

https://cloud.ibm.com/docs/watsonxdata?topic=watsonxdata-supported_sql_statements

83. Catalog Management:

https://cloud.ibm.com/docs/watsonxdata?topic=watsonxdata-reg_database

84. Table Format Support:

<https://cloud.ibm.com/docs/watsonxdata?topic=watsonxdata-supported-table-formats>

Vector Database & Embeddings

Milvus Advanced Topics

85. Index Building: https://milvus.io/docs/build_index.md

86. Search Parameters: <https://milvus.io/docs/search.md#Prepare-search-parameters>

87. Scalar Filtering: <https://milvus.io/docs/boolean.md>

88. Collection Operations: <https://milvus.io/docs/manage-collections.md>

89. Partition Management: <https://milvus.io/docs/manage-partitions.md>

90. Field Schema: <https://milvus.io/docs/schema.md#Field-schema>

Embedding Models

91. IBM slate-125m-english-rtrvr-v2:

<https://datapatform.cloud.ibm.com/docs/content/wsj/analyze-data/fm-model-embedding.html>

92. Sentence Transformers: <https://www.sbert.net/>

93. HuggingFace Embeddings:

https://huggingface.co/docs/transformers/main_classes/model#transformers.PreTrainedModel

Database & Storage

Iceberg Tables

94. Iceberg Python API: <https://py.iceberg.apache.org/>

95. Iceberg Partitioning: <https://iceberg.apache.org/docs/latest/partitioning/>

96. Iceberg Schema Evolution: <https://iceberg.apache.org/docs/latest/evolution/>

Presto/Trino SQL

97. Presto SQL Syntax: <https://prestodb.io/docs/current/sql.html>

98. Presto Date/Time Functions: <https://prestodb.io/docs/current/functions/datetime.html>

99. Presto String Functions: <https://prestodb.io/docs/current/functions/string.html>

100. Presto REST API Specification:

<https://prestodb.io/docs/current/develop/client-protocol.html>

IBM Cloud Object Storage

101. Bucket Configuration:

<https://cloud.ibm.com/docs/cloud-object-storage?topic=cloud-object-storage-endpoints>

102. Access Control:

<https://cloud.ibm.com/docs/cloud-object-storage?topic=cloud-object-storage-iam>

103. Storage Classes:

<https://cloud.ibm.com/docs/cloud-object-storage?topic=cloud-object-storage-classes>

- 122. Node.js ES Modules: <https://nodejs.org/api/esm.html>
- 123. package.json type field: <https://nodejs.org/api/packages.html#type>

CORS Issues

- 124. MDN CORS Guide: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>
- 125. Express CORS Configuration:
<https://expressjs.com/en/resources/middleware/cors.html>

Data Processing Libraries

Text Processing

- 126. re (Regular Expressions): <https://docs.python.org/3/library/re.html>
- 127. string Module: <https://docs.python.org/3/library/string.html>
- 128. textwrap: <https://docs.python.org/3/library/textwrap.html>

File Handling

- 129. pathlib: <https://docs.python.org/3/library/pathlib.html>
- 130. os Module: <https://docs.python.org/3/library/os.html>
- 131. shutil: <https://docs.python.org/3/library/shutil.html>
- 132. Node.js fs Module: <https://nodejs.org/api/fs.html>
- 133. Node.js path Module: <https://nodejs.org/api/path.html>

JSON & Data Serialization

- 134. Python json Module: <https://docs.python.org/3/library/json.html>
- 135. JavaScript JSON:
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON
- 136. JSON Schema Validation: <https://json-schema.org/>

HTTP & REST APIs

- 137. REST API Best Practices: <https://restfulapi.net/>
- 138. HTTP Status Codes: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>
- 139. HTTP Headers: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers>
- 140. Content-Type Headers:
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Type>

Authentication & Security

- 141. IBM Cloud API Key Management:
<https://cloud.ibm.com/docs/account?topic=account-userapikey>

- 142. Bearer Token Authentication: <https://datatracker.ietf.org/doc/html/rfc6750>
- 143. Environment Variables: <https://nodejs.org/en/learn/command-line/how-to-read-environment-variables-from-nodejs>

Testing & Validation

- 144. curl Documentation: <https://curl.se/docs/>
- 145. Python unittest: <https://docs.python.org/3/library/unittest.html>

- 146. Vector Similarity Search: <https://milvus.io/docs/metric.md>
- 147. COSINE Similarity: https://en.wikipedia.org/wiki/Cosine_similarity
- 148. IVF_FLAT Index: https://milvus.io/docs/index.md#IVF_FLAT
- 149. Semantic Search Concepts: <https://www.pinecone.io/learn/semantic-search/>

Document Information

Version: 1.0

Last Updated: December 2025

Maintained by: Team 06, Northeastern University

Contact: ummaraali2020@gmail.com

Change History

Version	Description
v1.0 (Dec 2025)	Initial release

— End of Document —